Supervisor Diagram Defence Brief

AI-Driven Weather Forecasting and Crop Recommendation System for Anuradhapura District

How to answer when the supervisor points to a diagram

Use this order: Purpose -> Main elements -> Flow/relationships -> Why this design was chosen -> Current implementation status.

Keep each first answer to about 30-60 seconds. Give more detail only if the supervisor asks a follow-up question. Most importantly, distinguish between a finalized design and a fully implemented feature.

Design view	What it proves to the supervisor
1. Use Case Diagram	Who uses the system and which functions belong to each role
2. System Architecture Diagram	How Angular, ASP.NET Core, SQL Server and Python/LSTM fit together
3. Entity Relationship Diagram (ERD)	How persistent project data is structured and related
4. Class Diagram	How core software/domain objects are structured
5. Sequence Diagram	How components communicate during a request
6. Activity / Workflow Diagram	The step-by-step decision and processing flow
7. Deployment Diagram	Where major software components run and communicate
8. Data Flow / System Flow Diagram	How weather, configuration, recommendations and validation data move

Safe status statement: “These diagrams represent the finalized system design. The database foundation, authentication/RBAC, administration foundation and standalone forecasting model have been implemented. Forecast-model backend integration and the end-to-end recommendation workflow are the next development stages.”

1. Use Case Diagram

What it shows: Shows the functional boundary of the system from the users' perspective.

Main elements: Public User, Registered User, Agricultural Officer and Administrator. Public users can access core weather/recommendation information without an account; registered users can authenticate and access recommendation history; officers review selected recommendations; administrators manage approved configuration and users.

> 30-second explanation:
> “This diagram defines who can interact with each system function. I separated public access from authenticated functionality and used role-based access for Registered Users, Agricultural Officers and Administrators. It helps convert the requirements into clear functional responsibilities.”

Likely question: Why do you need four user roles?

Suggested answer: Because their responsibilities are different. Public access supports basic accessibility, Registered Users need personal history, Agricultural Officers validate selected recommendations, and Administrators manage configuration and access control.

Current status: Requirements finalized. Authentication/RBAC and administrator backend foundation are implemented; the complete recommendation and officer-validation flow comes later.

2. System Architecture Diagram

What it shows: Shows the high-level technical structure and communication boundaries.

Main elements: Angular frontend -> ASP.NET Core Web API -> SQL Server, with a separate Python forecasting component containing the trained LSTM model. The backend remains the main application boundary and coordinates data access, business logic and forecasting integration.

> 30-second explanation:
> “I used a layered web architecture. Angular handles the user interface, ASP.NET Core provides the application and API layer, SQL Server stores persistent data, and Python contains the LSTM forecasting component. Keeping the forecasting model behind a clean integration boundary makes the system easier to maintain and test.”

Likely question: Why not call the Python model directly from Angular?

Suggested answer: The frontend should not own model integration or business rules. Routing requests through the backend keeps validation, authorization, business logic and model communication controlled in one application boundary.

Current status: Project structure exists and the standalone model is complete. The Python-to-backend integration is the immediate next phase.

3. Entity Relationship Diagram (ERD)

What it shows: Represents the SQL Server data model and relationships needed by the approved scope.

Main elements: Core entities include UserRole, User, Crop, CropEnvironmentalRequirement, SoilCompatibility, SuitabilityConfiguration, ForecastRecord, Recommendation, RecommendationCrop and RecommendationValidation.

> 30-second explanation:
> “The ERD translates the system requirements into persistent data entities. It separates user/role data, crop configuration, forecast records, recommendation results and officer validation so that each concern can be stored and queried clearly.”

Likely question: Why is there no separate RecommendationHistory table?

Suggested answer: History can be derived from the Recommendation records themselves, so a separate history table would duplicate data without adding a clear requirement.

Current status: Database foundation and the approved ten-entity schema are implemented and frozen.

4. Class Diagram

What it shows: Shows the static software/domain structure rather than only database tables.

Main elements: Represents domain entities and their relationships, with application services handling authentication, administration, forecasting integration and recommendation logic. Controllers are kept thin and business logic is separated from data access.

30-second explanation:
“The class diagram shows the software structure behind the system. The important design decision is separation of concerns: controllers expose endpoints, application services coordinate use cases, domain classes represent the core data/business concepts, and infrastructure handles persistence and external integration.”

Likely question: Is the class diagram the same as the ERD?

Suggested answer: No. The ERD focuses on database entities and relationships, while the class diagram represents software objects, responsibilities and relationships in the application design.

Current status: Core backend projects, domain entities and several services already exist. Forecast integration and recommendation services are still to be completed.

5. Sequence Diagram

What it shows: Shows the time-ordered interaction between the user interface, backend, forecasting component, recommendation logic and database for an important use case.

Main elements: A typical flow is: user request -> Angular -> ASP.NET API -> required data/configuration -> forecasting component -> suitability/recommendation processing -> persistence -> response to the frontend.

> 30-second explanation:
> “The sequence diagram focuses on interaction order. It shows which component calls the next component and when data is returned. This is useful for checking that the frontend does not bypass the backend and that forecasting and recommendation processing remain separate responsibilities.”

Likely question: Why separate forecasting from recommendation processing?

Suggested answer: The LSTM predicts future weather variables. The recommendation engine uses forecast evidence together with approved crop and soil configuration to calculate suitability. They solve different problems and can therefore be evaluated independently.

Current status: The design flow is finalized. The forecasting model exists, but the full sequence will become executable after Phase 6 integration and the recommendation phase.

6. Activity / Recommendation Workflow Diagram

What it shows: Shows the step-by-step business process and decision points for generating and, where applicable, validating a recommendation.

Main elements: Obtain required weather evidence -> generate 7-day forecast -> obtain approved crop/environmental and soil compatibility configuration -> calculate suitability -> classify and explain results -> save recommendation when applicable -> optional officer review with Pending Validation / Validated / Needs Review status.

> 30-second explanation:
> “This diagram explains the business workflow rather than component calls. The key point is that the LSTM produces weather forecasts, while the separate explainable suitability engine combines the approved factors to produce crop suitability categories and explanations.”

Likely question: Is the crop recommendation itself produced by the LSTM?

Suggested answer: No. The LSTM is only the weather forecasting model. Crop suitability is intentionally handled by a separate configurable weighted/rule-based engine using rainfall, temperature, humidity and soil compatibility.

Current status: Forecasting is complete. Suitability/recommendation processing and the full officer-validation workflow are later development phases.

7. Deployment Diagram

What it shows: Shows the runtime/deployment view of the solution and the boundaries between the web client, backend, database and forecasting service.

Main elements: Browser/client running Angular; ASP.NET Core API; SQL Server; Python forecasting runtime with the saved .keras model and scaler. Communication should occur through defined service/API boundaries.

> 30-second explanation:
> “The deployment diagram shows where the main components execute rather than their internal classes. The web client communicates with the backend, the backend accesses SQL Server and communicates with the Python forecasting component. This keeps the machine-learning runtime isolated from the user interface.”

Likely question: Which cloud provider are you deploying to?

Suggested answer: A specific deployment provider has not been frozen as part of the MSc scope yet. The design keeps the components deployable through standard boundaries without tying the project to one provider.

Current status: Logical deployment design is defined; a specific production hosting provider is intentionally not claimed as finalized.

8. Data Flow / Overall System Flow Diagram

What it shows: Shows how the main information moves through the system from inputs and configuration to forecasts, recommendations, storage and validation.

Main elements: Historical/model data supports the forecasting model; forecast outputs provide rainfall, temperature and humidity evidence; SQL Server stores configuration and system records; the recommendation engine consumes forecast/configuration data; results are presented to users and selected recommendations can be reviewed by an Agricultural Officer.

> 30-second explanation:
> “This diagram gives the end-to-end information view. It makes clear that weather forecasting is an input to the recommendation process, not the final recommendation itself, and that configuration and validation data are persisted separately.”

Likely question: What is the main data dependency of the recommendation engine?

Suggested answer: It depends on the forecasted rainfall, temperature and humidity plus approved crop environmental requirements, soil compatibility and configurable suitability weights/thresholds.

Current status: The data model and forecasting artifacts exist. End-to-end data movement will be completed through backend integration and the recommendation phases.

High-Probability Cross-Diagram Questions

Q: How do these diagrams connect to each other?

A: The Use Case Diagram defines user functionality; the Architecture Diagram defines the major technical components; the ERD defines persistent data; the Class Diagram defines software structure; the Sequence Diagram shows component interactions over time; the Activity Diagram shows business workflow; the Deployment Diagram shows runtime placement; and the Data Flow/System Flow view shows how information moves end to end.

Q: Which diagram is most closely connected to the actual implementation?

A: They all support implementation from different perspectives. At the current stage, the architecture and ERD are already strongly reflected in the project structure and database foundation, while the sequence/activity flows will become fully executable as forecast integration and recommendation development are completed.

Q: What is the role of AI in your system?

A: AI is used specifically for seven-day weather forecasting through the LSTM model. The agricultural recommendation is kept explainable through a separate configurable suitability engine rather than treating the whole system as one black-box AI model.

Q: Why is Agricultural Officer validation not automatic?

A: The officer is a read/review role for selected recommendations. The system provides forecast and suitability evidence, while the officer can record a validation status and optional comment. The officer does not edit the system configuration.

Q: Can an ordinary user become an Administrator or Agricultural Officer during registration?

A: No. Normal registration creates a Registered User. Privileged roles are controlled administratively; users cannot self-assign officer or administrator access.

Q: Does the design include features such as pest prediction, market prices or automated irrigation?

A: No. Those are outside the approved MSc scope. The recommendation core is limited to rainfall, temperature, humidity and soil compatibility for the six approved crops.

Final meeting reminder

Do not claim: whole system complete, exact district-wide microclimate coverage, production cloud deployment, or features outside the approved scope.

Do say: the design is finalized; database/auth/admin foundations and the standalone LSTM forecasting model are implemented; the next work is backend model integration, suitability/recommendation implementation, officer validation, Angular interfaces and system testing.